GUÍA LABORATORIO 04

Apache netbeans + spring boot+ arquitectura hexagonal + MySQL

Sistema Pedidos: login con sesiones con validación spring boot + encriptación

Control de entrada y salida

Ingreso de empleado

Registro de clientes

Ing Natividad Caceres

Ing Elard Rodriguez Marca

1. Objetivo

Implementar un módulo web conectado a la base de datos Pedidos, con autenticación mediante Spring Security, gestión de sesiones HTTP, contraseñas almacenadas con BCrypt y formularios de registro de empleados y clientes. El código entregado es una base compacta para el aula; al final se proponen retos para completar la separación estricta de responsabilidades de la arquitectura hexagonal.

2. Tecnologías y supuestos

- Java 17, Spring Boot 3.x, Maven y Apache NetBeans.
- MySQL como motor de base de datos.
- Spring Web, Spring Security, Spring Data JPA, Thymeleaf y Bean Validation.
- Para evitar asumir la estructura interna de la base Pedidos, el ejemplo crea tablas separadas: usuarios_app, empleados_app y clientes_app.

3. Estructura de carpetas

Estructura sugerida

```
PedidosHexagonal/
├── pom.xml
└── src/main/
    ├── java/com/empresa/pedidos/
    │   ├── PedidosApplication.java
    │   ├── domain/model/
    │   │   └── Usuario.java
    │   └── infrastructure/
    │       ├── persistence/entity/
    │       │   ├── UsuarioEntity.java
    │       │   ├── Empleado.java
    │       │   └── Cliente.java
    │       ├── persistence/repository/
    │       │   ├── UsuarioJpaRepository.java
    │       │   ├── EmpleadoJpaRepository.java
    │       │   └── ClienteJpaRepository.java
    │       ├── security/
    │       │   ├── SecurityConfig.java
    │       │   └── AdminInicial.java
    │       └── web/
    │           ├── LoginController.java
    │           ├── EmpleadoController.java
    │           └── ClienteController.java
```

```
    └── resources/
        ├── application.properties
        └── templates/
            ├── login.html
            ├── menu.html
            ├── empleados.html
            └── clientes.html
```

4. Configuración Maven

Archivo: pom.xml

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.empresa</groupId>
    <artifactId>PedidosHexagonal</artifactId>
    <version>1.0.0</version>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.5</version>
        <relativePath/>
    </parent>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

5. Configurar la conexión

Archivo: src/main/resources/application.properties

```
spring.application.name=PedidosHexagonal
spring.datasource.url=jdbc:mysql://localhost:3306/Pedidos
spring.datasource.username=root
spring.datasource.password=TU_CLAVE

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.open-in-view=false

server.port=8080
server.servlet.session.timeout=15m
server.servlet.session.cookie.http-only=true
server.servlet.session.cookie.same-site=lax
```

Nota: ddl-auto=update se usa solo para facilitar la práctica local. Revise el esquema antes de ejecutar; en ambientes compartidos o productivos, use scripts de migración controlados.

6. Clase principal

PedidosApplication.java · paquete com.empresa.pedidos

```
package com.empresa.pedidos;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class PedidosApplication {
    public static void main(String[] args) {
        SpringApplication.run(PedidosApplication.class, args);
    }
}
```

7. Modelo de dominio Usuario

Usuario.java · paquete domain.model

```
package com.empresa.pedidos.domain.model;

public class Usuario {
    private Long id;
    private String username;
    private String passwordHash;
    private String rol;
    private boolean activo;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPasswordHash() { return passwordHash; }
    public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }
    public String getRol() { return rol; }
    public void setRol(String rol) { this.rol = rol; }
    public boolean isActivo() { return activo; }
    public void setActivo(boolean activo) { this.activo = activo; }
}
```

8. Entidades JPA

UsuarioEntity.java · paquete infrastructure.persistence.entity

```
package com.empresa.pedidos.infrastructure.persistence.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "usuarios_app")
public class UsuarioEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 80)
    private String username;

    @Column(nullable = false, length = 100)
    private String passwordHash;

    @Column(nullable = false, length = 30)
    private String rol = "EMPLEADO";

    @Column(nullable = false)
    private boolean activo = true;

    public Long getId() { return id; }
    public String getUsername() { return username; }
    public String getPasswordHash() { return passwordHash; }
    public String getRol() { return rol; }
    public boolean isActivo() { return activo; }
    public void setId(Long id) { this.id = id; }
    public void setUsername(String username) { this.username = username; }
    public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }
    public void setRol(String rol) { this.rol = rol; }
    public void setActivo(boolean activo) { this.activo = activo; }
}
```

Empleado.java · paquete infrastructure.persistence.entity

```
package com.empresa.pedidos.infrastructure.persistence.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

@Entity
@Table(name = "empleados_app")
public class Empleado {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Size(max = 100)
    private String nombres;

    @NotBlank
    @Size(max = 100)
    private String apellidos;

    @NotBlank
    @Size(max = 20)
    @Column(unique = true)
    private String documento;
```

```
    @Email
    @Size(max = 150)
    private String correo;

    public Long getId() { return id; }
    public String getNombres() { return nombres; }
    public void setNombres(String nombres) { this.nombres = nombres; }
    public String getApellidos() { return apellidos; }
    public void setApellidos(String apellidos) { this.apellidos = apellidos; }
    public String getDocumento() { return documento; }
    public void setDocumento(String documento) { this.documento = documento; }
    public String getCorreo() { return correo; }
    public void setCorreo(String correo) { this.correo = correo; }
}
```

Cliente.java · paquete infrastructure.persistence.entity

```
package com.empresa.pedidos.infrastructure.persistence.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

@Entity
@Table(name = "clientes_app")
public class Cliente {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Size(max = 150)
    private String nombre;

    @NotBlank
    @Size(max = 20)
    @Column(unique = true)
    private String documento;

    @Size(max = 200)
    private String direccion;

    @Email
    @Size(max = 150)
    private String correo;

    public Long getId() { return id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getDocumento() { return documento; }
    public void setDocumento(String documento) { this.documento = documento; }
    public String getDireccion() { return direccion; }
    public void setDireccion(String direccion) { this.direccion = direccion; }
    public String getCorreo() { return correo; }
    public void setCorreo(String correo) { this.correo = correo; }
}
```

9. Repositorios Spring Data JPA

UsuarioJpaRepository.java

```
package com.empresa.pedidos.infrastructure.persistence.repository;

import com.empresa.pedidos.infrastructure.persistence.entity.UsuarioEntity;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
```

```
public interface UsuarioJpaRepository extends JpaRepository<UsuarioEntity, Long> {
    Optional<UsuarioEntity> findByUsername(String username);
}
```

EmpleadoJpaRepository.java

```
package com.empresa.pedidos.infrastructure.persistence.repository;

import com.empresa.pedidos.infrastructure.persistence.entity.Empleado;
import org.springframework.data.jpa.repository.JpaRepository;

public interface EmpleadoJpaRepository extends JpaRepository<Empleado, Long> {
    boolean existsByDocumento(String documento);
}
```

ClienteJpaRepository.java

```
package com.empresa.pedidos.infrastructure.persistence.repository;

import com.empresa.pedidos.infrastructure.persistence.entity.Cliente;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ClienteJpaRepository extends JpaRepository<Cliente, Long> {
    boolean existsByDocumento(String documento);
}
```

10. Seguridad: login, BCrypt, roles y sesiones

SecurityConfig.java

```
package com.empresa.pedidos.infrastructure.security;
import com.empresa.pedidos.infrastructure.persistence.entity.UsuarioEntity;
import com.empresa.pedidos.infrastructure.persistence.repository.UsuarioJpaRepository;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.core.userdetails.*;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;

@Configuration
public class SecurityConfig {
   @Bean
   PasswordEncoder passwordEncoder() {
      return new BCryptPasswordEncoder();
   }

   @Bean
   UserDetailsService userDetailsService(UsuarioJpaRepository repository) {
      return username -> repository.findByUsername(username)
         .filter(u -> u.isActivo())
         .map(u -> User.withUsername(u.getUsername())
            .password(u.getPasswordHash())
            .roles(u.getRol())
            .build())
         .orElseThrow(() ->
            new UsernameNotFoundException("Usuario no válido"));
```

```
    }

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/login", "/error").permitAll()
                .requestMatchers("/empleados/**").hasRole("ADMIN")
                .requestMatchers("/clientes/**").hasAnyRole("ADMIN", "EMPLEADO")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .defaultSuccessUrl("/menu", true)
                .failureUrl("/login?error")
                .permitAll()
            )
            .logout(logout -> logout
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login?logout")
                .invalidateHttpSession(true)
                .clearAuthentication(true)
                .deleteCookies("JSESSIONID")
                .permitAll()
            )
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
                .sessionFixation(fixation -> fixation.changeSessionId())
            );
        return http.build();
    }
}
```

11. Controlador de navegación

LoginController.java

```
package com.empresa.pedidos.infrastructure.web;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class LoginController {
    @GetMapping("/login")
    public String login() { return "login"; }

    @GetMapping("/menu")
    public String menu() { return "menu"; }
}
```

12. Vistas Thymeleaf

templates/login.html

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Acceso al sistema Pedidos</title>
</head>
<body>
    <h2>Login - Sistema Pedidos</h2>
    <p th:if="${param.error}">Usuario o contraseña incorrectos.</p>
    <p th:if="${param.logout}">La sesión se cerró correctamente.</p>

    <form th:action="@{/login}" method="post">
        <label>Usuario:</label>
        <input type="text" name="username" required maxlength="80">
        <br><br>
        <label>Contraseña:</label>
        <input type="password" name="password" required>
        <br><br>
        <button type="submit">Ingresar</button>
    </form>
</body>
</html>
```

templates/menu.html

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Menú principal</title>
</head>
<body>
    <h1>Sistema Pedidos</h1>
    <p>Bienvenido, <strong th:text="${#authentication.name}"></strong></p>
    <nav>
        <a th:href="@{/empleados/nuevo}">Registrar empleado</a>
        <br><br>
        <a th:href="@{/clientes/nuevo}">Registrar cliente</a>
    </nav>
    <br>
    <form th:action="@{/logout}" method="post">
        <button type="submit">Cerrar sesión</button>
    </form>
</body>
</html>
```

13. Controladores de registro

EmpleadoController.java

```
package com.empresa.pedidos.infrastructure.web;

import com.empresa.pedidos.infrastructure.persistence.entity.Empleado;
import com.empresa.pedidos.infrastructure.persistence.repository.EmpleadoJpaRepository;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/empleados")
public class EmpleadoController {
    private final EmpleadoJpaRepository repository;
```

```
    public EmpleadoController(EmpleadoJpaRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/nuevo")
    public String nuevo(Model model) {
        model.addAttribute("empleado", new Empleado());
        return "empleados";
    }

    @PostMapping("/guardar")
    public String guardar(@Valid @ModelAttribute("empleado") Empleado empleado,
                          BindingResult result, Model model) {
        if (result.hasErrors()) return "empleados";

        empleado.setDocumento(empleado.getDocumento().trim());
        if (repository.existsByDocumento(empleado.getDocumento())) {
            model.addAttribute("mensaje", "El documento ya se encuentra registrado.");
            return "empleados";
        }

        repository.save(empleado);
        model.addAttribute("mensaje", "Empleado registrado correctamente.");
        model.addAttribute("empleado", new Empleado());
        return "empleados";
    }
}
```

ClienteController.java

```
package com.empresa.pedidos.infrastructure.web;

import com.empresa.pedidos.infrastructure.persistence.entity.Cliente;
import com.empresa.pedidos.infrastructure.persistence.repository.ClienteJpaRepository;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/clientes")
public class ClienteController {
    private final ClienteJpaRepository repository;

    public ClienteController(ClienteJpaRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/nuevo")
    public String nuevo(Model model) {
        model.addAttribute("cliente", new Cliente());
        return "clientes";
    }

    @PostMapping("/guardar")
    public String guardar(@Valid @ModelAttribute("cliente") Cliente cliente,
                          BindingResult result, Model model) {
        if (result.hasErrors()) return "clientes";

        cliente.setDocumento(cliente.getDocumento().trim());
        if (repository.existsByDocumento(cliente.getDocumento())) {
            model.addAttribute("mensaje", "El documento ya se encuentra registrado.");
            return "clientes";
        }

        repository.save(cliente);
        model.addAttribute("mensaje", "Cliente registrado correctamente.");
        model.addAttribute("cliente", new Cliente());
```

```
        return "clientes";
    }
}
```

templates/empleados.html

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head><meta charset="UTF-8"><title>Registro de empleados</title></head>
<body>
    <h2>Registrar empleado</h2>
    <p th:if="${mensaje}" th:text="${mensaje}"></p>
    <form th:action="@{/empleados/guardar}" th:object="${empleado}" method="post">
        <label>Nombres:</label>
        <input type="text" th:field="*{nombres}" required maxlength="100">
        <p th:errors="*{nombres}"></p>
        <label>Apellidos:</label>
        <input type="text" th:field="*{apellidos}" required maxlength="100">
        <p th:errors="*{apellidos}"></p>
        <label>Documento:</label>
        <input type="text" th:field="*{documento}" required maxlength="20">
        <p th:errors="*{documento}"></p>
        <label>Correo:</label>
        <input type="email" th:field="*{correo}" maxlength="150">
        <p th:errors="*{correo}"></p>
        <button type="submit">Guardar empleado</button>
    </form>
    <a th:href="@{/menu}">Volver al menú</a>
</body>
</html>
```

templates/clientes.html

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head><meta charset="UTF-8"><title>Registro de clientes</title></head>
<body>
    <h2>Registrar cliente</h2>
    <p th:if="${mensaje}" th:text="${mensaje}"></p>
    <form th:action="@{/clientes/guardar}" th:object="${cliente}" method="post">
        <label>Nombre o razón social:</label>
        <input type="text" th:field="*{nombre}" required maxlength="150">
        <p th:errors="*{nombre}"></p>
        <label>Documento:</label>
        <input type="text" th:field="*{documento}" required maxlength="20">
        <p th:errors="*{documento}"></p>
        <label>Dirección:</label>
        <input type="text" th:field="*{direccion}" maxlength="200">
        <p th:errors="*{direccion}"></p>
        <label>Correo:</label>
        <input type="email" th:field="*{correo}" maxlength="150">
        <p th:errors="*{correo}"></p>
        <button type="submit">Guardar cliente</button>
    </form>
    <a th:href="@{/menu}">Volver al menú</a>
</body>
</html>
```

14. Crear un administrador inicial (solo para el aula)

AdminInicial.java

```
package com.empresa.pedidos.infrastructure.security;

import com.empresa.pedidos.infrastructure.persistence.entity.UsuarioEntity;
import com.empresa.pedidos.infrastructure.persistence.repository.UsuarioJpaRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.password.PasswordEncoder;
```

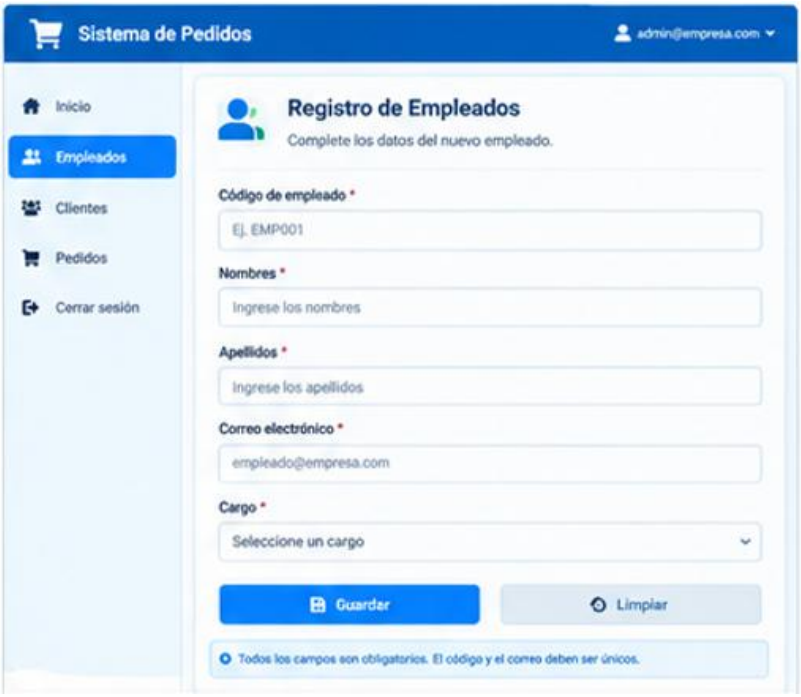
```
@Configuration
public class AdminInicial {
    @Bean
    CommandLineRunner crearAdmin(UsuarioJpaRepository repository,
                                 PasswordEncoder encoder) {
        return args -> {
            if (repository.findByUsername("admin").isEmpty()) {
                UsuarioEntity u = new UsuarioEntity();
                u.setUsername("admin");
                u.setPasswordHash(encoder.encode("Admin123*"));
                u.setRol("ADMIN");
                u.setActivo(true);
                repository.save(u);
            }
        };
    }
}
```

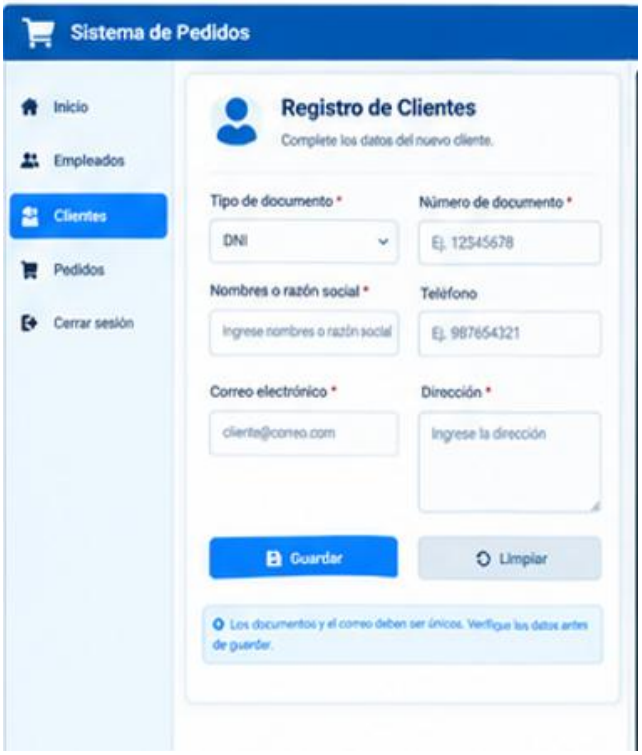
Credenciales locales de prueba: usuario admin; contraseña Admin123*. No utilizar estas credenciales en producción. Elimine el inicializador después de crear la cuenta.

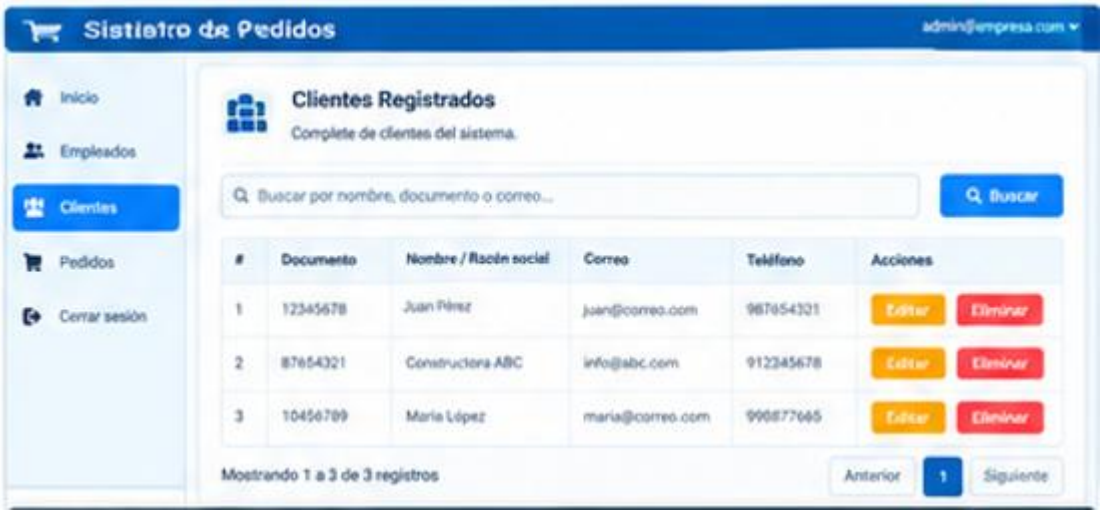
Formularios propuestos






Sistema de Pedidos
admin@empresa.com
Inicio
Empleados
Clientes
Pedidos
Cerrar sesión
Registro de Empleados
Complete los datos del nuevo empleado.
Código de empleado *
Ej. EMP001
Nombres *
Ingrese los nombres
Apellidos *
Ingrese los apellidos
Correo electrónico *
empleado@empresa.com
Cargo *
Seleccione un cargo
Guardar
Limpiar
Todos los campos son obligatorios. El código y el correo deben ser únicos.


Sistema de Pedidos
Inicio
Empleados
Clientes
Pedidos
Cerrar sesión
Registro de Clientes
Complete los datos del nuevo cliente.
Tipo de documento *
DNI
Número de documento *
Ej. 12545678
Nombres o razón social *
Ingrese nombres o razón social
Teléfono
Ej. 987654321
Correo electrónico *
cliente@correo.com
Dirección *
Ingrese la dirección
Guardar
Limpiar
Los documentos y el correo deben ser únicos. Verifique los datos antes de guardar.



15. Ejecución y pruebas

- Iniciar MySQL y verificar que exista la base de datos Pedidos.
- Configurar usuario y contraseña en application.properties.
- Ejecutar PedidosApplication.java desde NetBeans.
- Abrir http://localhost:8080/login e ingresar con las credenciales de prueba.
- Registrar un cliente y comprobar que se almacene en clientes_app.
- Ingresar a /empleados/nuevo con ADMIN y probar el registro.
- Cerrar sesión y comprobar que una ruta privada solicite autenticación.

16. Retos para completar la arquitectura hexagonal

- Crear puertos de salida para usuarios, empleados y clientes.
- Implementar adaptadores JPA que implementen esos puertos.
- Crear casos de uso RegistrarEmpleado y RegistrarCliente, trasladando allí las reglas de negocio.

17. Observaciones importantes

- El ejemplo es una base didáctica y compacta; los controladores de registro acceden directamente a los repositorios JPA. Por ello, todavía no constituye una implementación estricta de arquitectura hexagonal.
- La autorización configura ADMIN para empleados y ADMIN o EMPLEADO para clientes. Para crear cuentas con rol EMPLEADO se necesita un proceso administrativo controlado.
- Se mantiene CSRF habilitado. Los formularios POST de Thymeleaf deben conservar el token CSRF generado por Spring Security.
- Las validaciones del navegador no sustituyen las validaciones del servidor. En producción deben añadirse restricciones de base de datos, manejo centralizado de errores, HTTPS, pruebas automatizadas y gestión segura de secretos.
- La estructura de Pedidos puede diferir de este ejemplo. Ajuste nombres de tablas, columnas y relaciones antes de ejecutarlo contra una base existente.